Module 6

SystemVerilog Testbench Features

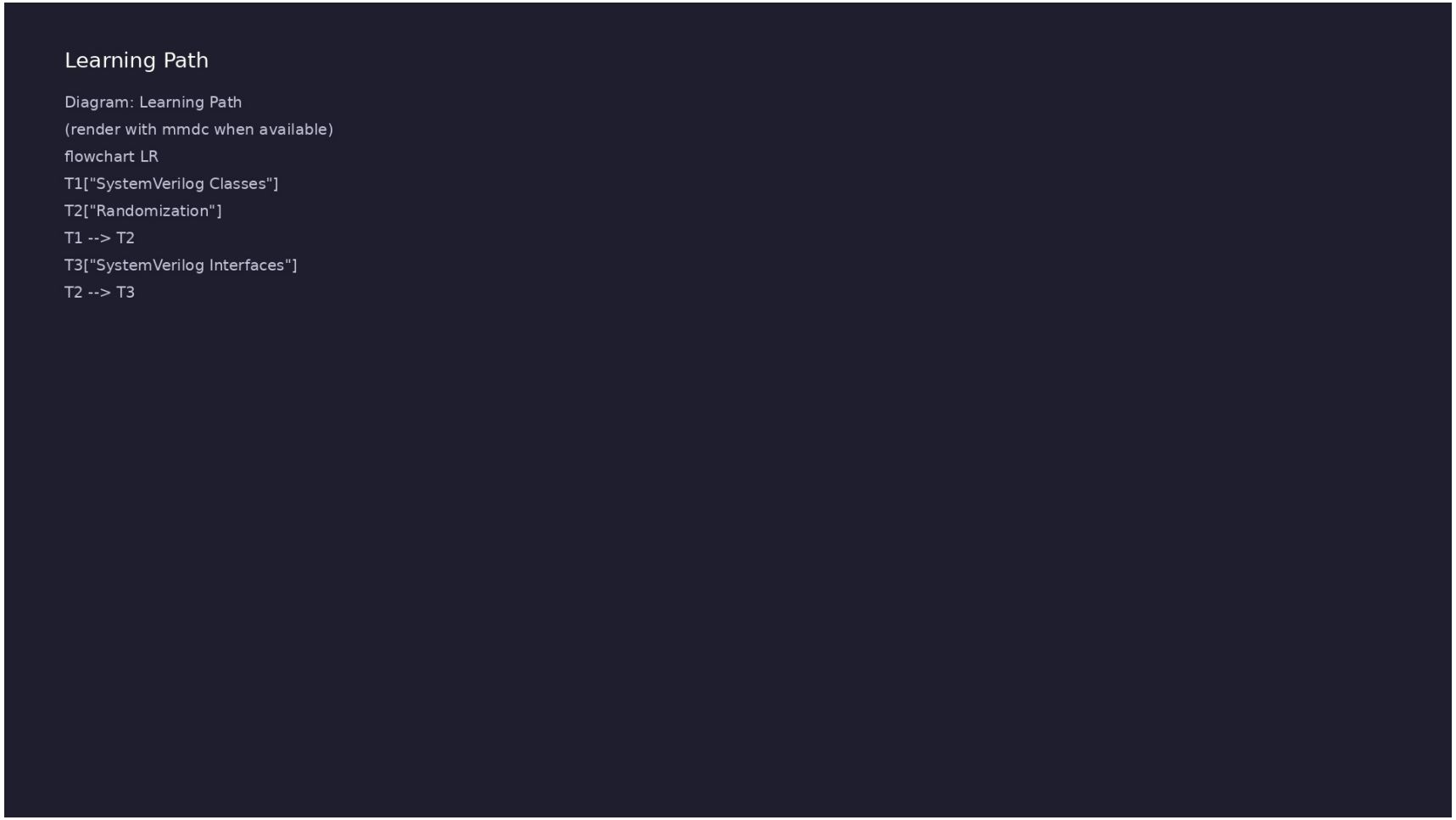
Learning objectives

- Master SystemVerilog features for advanced testbench development (primarily for iverilog, with Veri...

Prerequisites

- See module README

Learning path



Master SystemVerilog features for advanced testbench development (primarily for iverilog, with Verilator considerations)

Overview

- This module introduces SystemVerilog features that enhance testbench capabilities. You'll learn abo...

Design architecture

1. SystemVerilog testbench layer

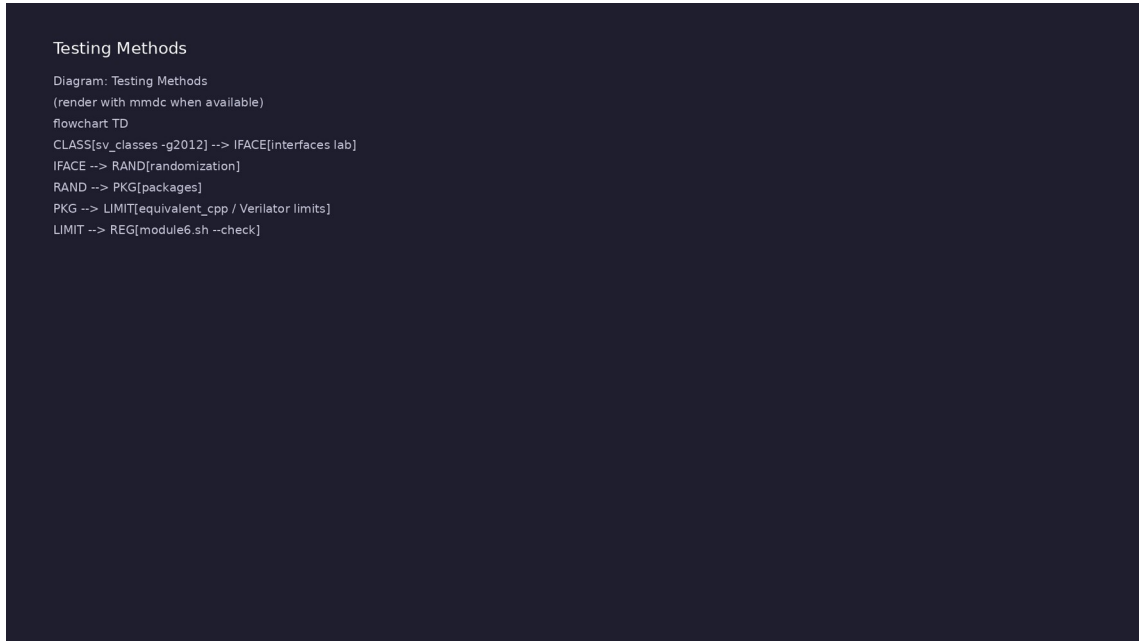
- Classes: Encapsulate transactions, drivers, and test state (`sv_classes` examples)
- Interfaces: Bundle DUT-facing signals; connect TB components via modports
- Packages: Shared types, parameters, and class declarations across compilation units
- Randomization: Constraint blocks on transaction fields for varied legal stimulus

2. DUT and tool considerations

- `dut/simple_gates/`, `dut/multiplexers/`: RTL exercised with SV TB features
- iverilog: Primary target for classes, interfaces, and `randomize()`
- Verilator: Limited SV — `equivalent_cpp/` documents C++ workarounds for unsupported constructs
- Compilation order: Package → interface → classes → TB top (see example Makefiles)

3. Object-oriented TB topology

- Transaction item: Data class holding addr, data, or opcode fields
- Driver class: `randomize()` item, drive interface signals, wait for handshake
- Test class: Builds environment, runs sequences, reports `uvm-style` summary without UVM libs



RTL block diagram (reference)

Module 6: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Module 6: stimulus, observation, and checking.

SV compile — iverilog -g2012

```
class_based_testbench: class_based_testbench.sv
	@echo "=== Compiling SystemVerilog class-based testbench
(iverilog) ==="
	$(IVERILOG) -g2012 -o class_based_testbench
class_based_testbench.sv $(DUT_DIR)/simple_gates/and_gate.v
	@echo "=== Running simulation ==="
	$(VVP) class_based_testbench
```

Transaction class definition

```
class transaction;
    // Randomizable input signals (can be randomized with randomize()
method)
    // rand keyword makes these variables eligible for constrained
randomization
    rand bit a;  // First input to AND gate
    rand bit b;  // Second input to AND gate

    // Non-randomizable output (calculated, not randomized)
    bit expected;  // Expected output value (calculated in
post_randomize)

    /**
     * Constructor - Called automatically when object is created
     *
     * Initializes all class properties to default values.
     * In SystemVerilog, new() is the constructor function.
```

Interface — signal bundle

```
/**
 * AND Gate Interface Definition
 *
 * This interface encapsulates all signals for the AND gate:
 * - Input signals: a, b
 * - Output signal: y
 *
 * Interfaces provide:
 * 1. Signal grouping: Related signals in one place
 * 2. Direction control: Modports define who can drive/read which
signals
 * 3. Reusability: Same interface can be used in multiple testbenches
 *
 * UVM Pattern: Similar to UVM virtual interfaces, which allow classes
 * to access RTL signals through interface handles.
 */
interface and_gate_if;
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator + UVM_HOME compile RTL, interface, and test_*.sv
- UVM phases: build → connect → run_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test_* (see module6 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

Directed test execution sequence (1)

- post_randomize() sets expected
- driver applies transaction
- compare before next randomize
- Package
- interface

Directed test execution sequence (2)

- class
- randomize()
- -g2012 compile

Interface-based TB build

```
# Makefile for interface examples

IVERILOG = iverilog
VVP = vvp
VERILATOR = verilator
DUT_DIR = ../../dut

all: interface_based_testbench interface_based_testbench_cpp

# SystemVerilog interface-based testbench (iverilog)
interface_based_testbench: interface_based_testbench.sv
	@echo "=== Compiling SystemVerilog interface-based testbench
(iverilog) ==="
	$(IVERILOG) -g2012 -o interface_based_testbench
interface_based_testbench.sv $(DUT_DIR)/simple_gates/and_gate.v
	@echo "=== Running simulation ==="
	$(VVP) interface_based_testbench
```

Test class run method

```
fail_count++;
            $error("[FAIL] Test %0d: a=%b, b=%b, result=%b,
expected=%b",
                   test_count, a, b, result, expected);
        end
    endfunction

    /**
     * Print Summary Method - Display test results summary
     *
     * Displays a formatted summary of all test results including:
     * - Total number of tests executed
     * - Number of passing tests
     * - Number of failing tests
     *
     * UVM Pattern: Similar to UVM's end_of_elaboration_phase() or
     * report_phase() where test results are summarized and reported.
```

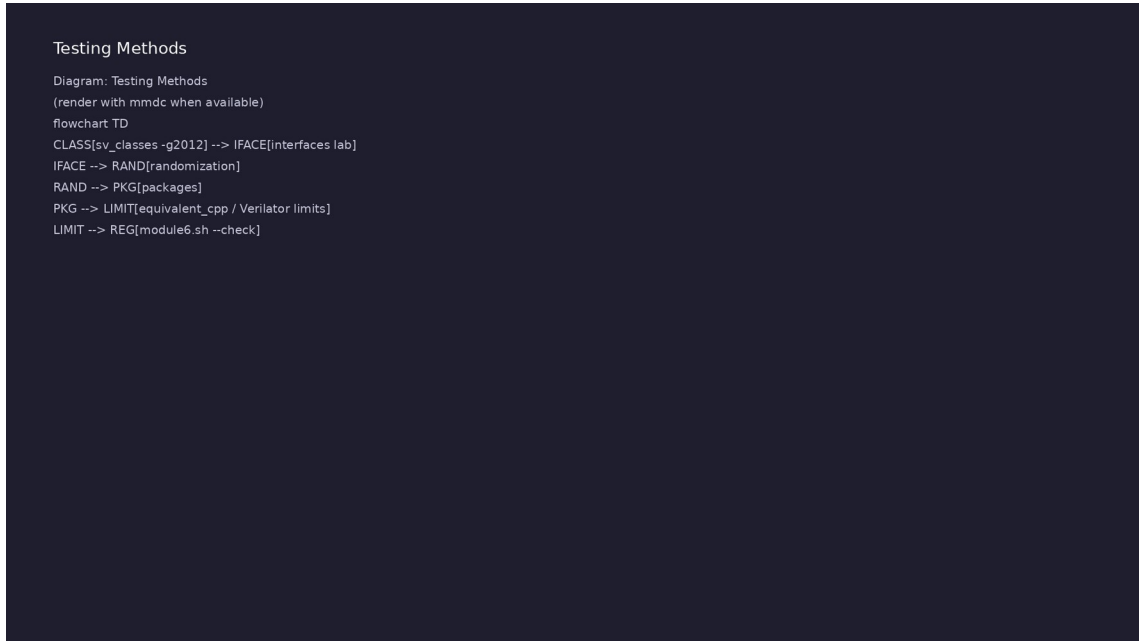
Constraint randomization

```
/**
 * Randomized Testbench - SystemVerilog (iverilog)
 *
 * This example demonstrates constrained randomization, a powerful
verification technique
 * inspired by UVM (Universal Verification Methodology). Constrained
randomization allows
 * you to generate large numbers of test vectors automatically while
ensuring they meet
 * specific requirements through constraints.
 *
 * Learning Objectives:
 * - Understand SystemVerilog randomization (rand, randc keywords)
 * - Learn constraint-based randomization (constraint blocks)
 * - Master post_randomize() callback for derived value calculation
 * - Understand seed control for reproducible random tests
 * - Learn random test generation strategies
```

Verification & testing methods

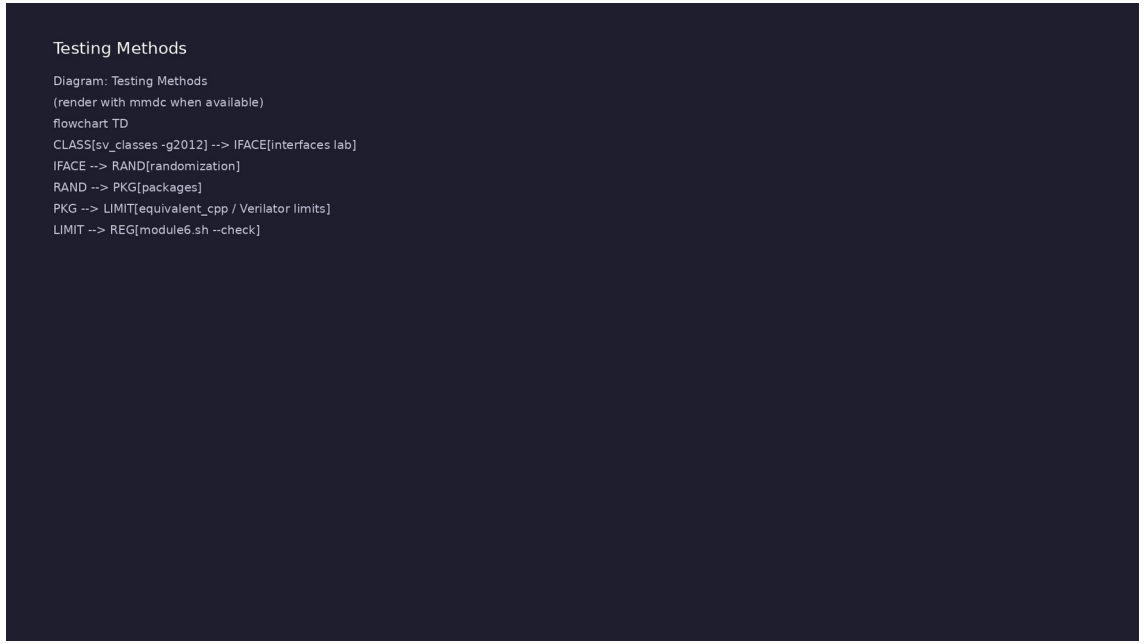
1. Constrained-random stimulus

- Constraints: Legal opcodes, address ranges, and timing gaps enforced in class
- Seeds: Repeatable runs with `srandom(seed)` for debug vs random exploration
- Coverage of space: Many transactions from few lines of sequence code



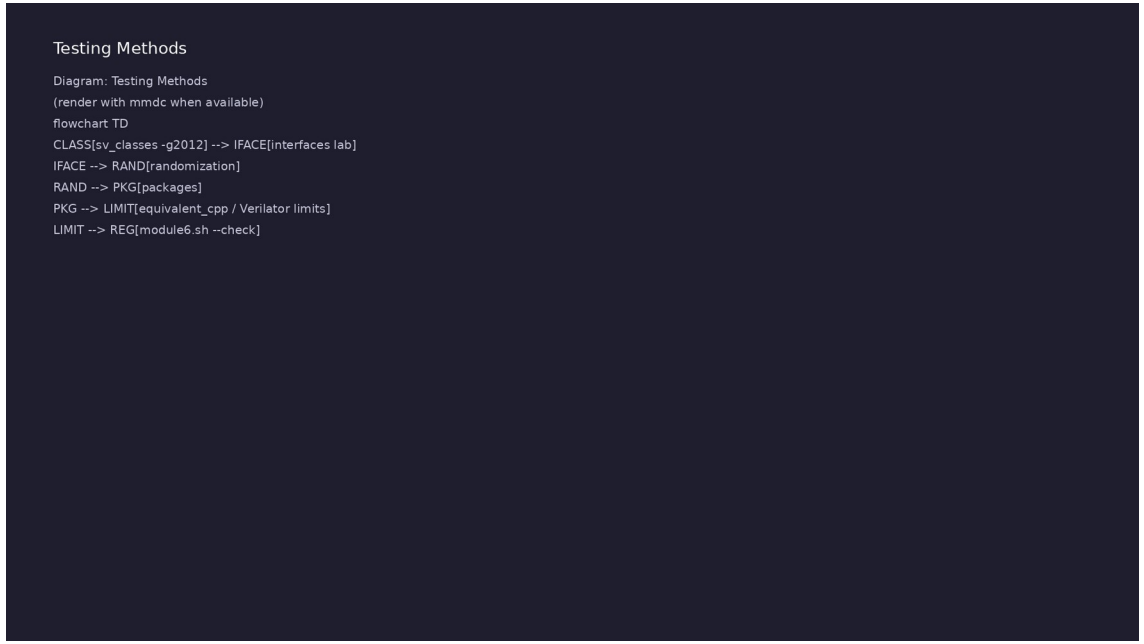
2. Interface-based checking

- Modports: Driver drives `DUT_MP`; monitor samples `MON_MP` on same interface
- Scoreboard hook: Monitor pushes observed transactions to checker class
- Encapsulation: DUT port list changes localized to interface definition



3. SV vs C++ equivalence verification

- Dual implementation: Compare SV class TB behavior to C++ equivalent where Verilator used
- Limitation tests: Document constructs that fail Verilator lint — plan tool split
- Regression: `./scripts/module6.sh` runs iverilog-first examples then C++ equivalents



Package import pattern

```
/**
 * Package Example - SystemVerilog (iverilog)
 *
 * This example demonstrates how to use SystemVerilog packages to
organize
 * and share definitions across multiple testbenches. It shows package
import,
 * usage of package-defined types, functions, and tasks.
 *
 * Learning Objectives:
 * - Understand package import syntax
 * - Learn how to use package-defined types
 * - Master package function and task usage
 * - Understand namespace management with packages
 *
 * UVM Concepts Demonstrated:
 * - Package organization: Similar to UVM's package structure
```

Syllabus topics

1. SystemVerilog Classes (1/6)

- SystemVerilog classes → UVM base classes (`uvm_object`, `uvm_component`, `uvm_sequence_item`)
- Transaction classes → UVM sequence items (data structures for verification)
- Testbench classes → UVM test classes (test orchestration)
- Class hierarchy → UVM component hierarchy
- Class Basics for Testbenches
- Class declaration syntax

1. SystemVerilog Classes (2/6)

- Class properties (data members)
- Class methods (functions and tasks)
- Class instantiation (object creation)
- Constructor (`new()` function)
- Class Methods and Properties
- Method declaration and implementation

1. SystemVerilog Classes (3/6)

- Property declaration and initialization
- Access modifiers (public, protected, local)
- Method overloading concepts
- Class Instantiation
- Object creation: `class_name obj = new();`
- Object lifetime management

1. SystemVerilog Classes (4/6)

- Automatic garbage collection
- Memory management best practices
- Object-Oriented Testbenches
- OOP principles in verification
- Encapsulation: Data and methods together
- Inheritance: Base classes and derived classes

1. SystemVerilog Classes (5/6)

- Polymorphism: Virtual methods and dynamic dispatch
- Class-Based Testbench Organization
- Transaction classes: Model test data
- Testbench classes: Orchestrate tests
- Class hierarchy design
- Reusability patterns

1. SystemVerilog Classes (6/6)

- Maintainability best practices
- `class_based_testbench.sv`: Comprehensive class-based testbench with detailed comments
- Transaction class with `post_randomize()` callback
- Testbench class for test orchestration
- Object instantiation and usage patterns
- `class_based_testbench_cpp.cpp`: C++ equivalent for Verilator

2. Randomization (1/7)

- SystemVerilog randomization → UVM's constrained random framework
- Constraint blocks → UVM constraint blocks in sequence items
- Random test generation → UVM sequence generation
- Seed control → UVM command-line seed control (`+seed`)
- Random Variable Generation
- `$random()`: Signed random integer

2. Randomization (2/7)

- `$urandom()`: Unsigned random integer
- `$urandom_range()`: Random value in range
- Seed control: `$urandom(seed)` for reproducibility
- Constrained Randomization Basics
- `rand` keyword: Random variable (can repeat)
- `randc` keyword: Cyclic random variable (no repeats until all used)

2. Randomization (3/7)

- Constraint blocks: Define valid value ranges
- Constraint solving: Automatic constraint satisfaction
- `randomize()` method: Generate random values
- Constraint Syntax
- Post-Randomization Callbacks
- `post_randomize()`: Called after successful randomization

2. Randomization (4/7)

- Calculate derived values based on randomized inputs
- UVM pattern: Similar to `post_randomize()` in UVM sequence items
- Random Test Generation
- Generate thousands of test vectors automatically
- Ensure test vectors meet design requirements
- Achieve better coverage with less manual effort

2. Randomization (5/7)

- Random sequences and patterns
- Seed Control
- Seed setting: `$urandom(seed)` for reproducible tests
- Seed management: Track seeds for debugging
- Reproducibility: Same seed = same test sequence
- Seed strategies: Random vs. fixed seeds

2. Randomization (6/7)

- Randomization Strategies
- Constraint design: Balance between flexibility and control
- Coverage-driven randomization: Guide randomization toward coverage goals
- Weighted distributions: Control probability of values
- Best practices: Keep constraints simple and maintainable
- `randomized_testbench.sv`: Comprehensive randomized testbench with:

2. Randomization (7/7)

- Constraint blocks for valid value ranges
- `post_randomize()` callback for derived values
- Seed control and reproducibility
- Random test generation loop
- `randomized_testbench_cpp.cpp`: C++ equivalent using `<random>` library

3. SystemVerilog Interfaces (1/6)

- SystemVerilog interfaces → UVM virtual interfaces
- Modports → UVM interface modports (driver, monitor, etc.)
- Interface-based testbenches → UVM testbench structure
- Virtual interfaces → UVM's way to connect classes to RTL
- Interface Declaration and Usage
- Interface syntax: `interface name; ... endinterface`

3. SystemVerilog Interfaces (2/6)

- Signal grouping: Related signals together
- Interface instantiation: `interface_name instance();`
- Interface connection: Pass to modules via modports
- Modports for Direction Control
- Modport syntax: `modport name(input sig1, output sig2);`
- Direction control: Define who can drive/read which signals

3. SystemVerilog Interfaces (3/6)

- Multiple modports: Different views for different users
- Type safety: Prevents incorrect signal connections
- Interface in Testbenches
- Interface-based testbenches: Clean signal organization
- DUT wrapping: Connect DUT to interface via wrapper
- Interface reusability: Same interface for multiple testbenches

3. SystemVerilog Interfaces (4/6)

- Interface best practices: Clear naming, proper modports
- Virtual Interfaces (Advanced)
- Virtual interface syntax: `virtual interface_name vif;`
- Virtual interface usage: Interface handles in classes
- Dynamic interfaces: Runtime interface assignment
- Virtual interface patterns: UVM-style class-to-RTL connection

3. SystemVerilog Interfaces (5/6)

- Interface-Based Testbenches
- Interface design: Group related signals
- Interface patterns: Common interface structures
- Interface organization: Hierarchical interfaces
- Interface best practices: Modports, naming, documentation
- `interface_based_testbench.sv`: Comprehensive interface-based testbench with:

3. SystemVerilog Interfaces (6/6)

- Interface definition with modports (dut, tb)
- DUT wrapper module
- Testbench using interface
- Detailed comments on modport usage
- `interface_based_testbench_cpp.cpp`: C++ equivalent using structs

4. Advanced Data Types (1/5)

- Structures and Unions
- Structure syntax
- Union syntax
- Packed structures
- Unpacked structures
- Enumerated Types

4. Advanced Data Types (2/5)

- Enum syntax
- Enum usage
- Enum values
- Enum best practices
- Dynamic Arrays
- Dynamic array syntax

4. Advanced Data Types (3/5)

- Dynamic array operations
- Dynamic array usage
- Dynamic array best practices
- Associative Arrays
- Associative array syntax
- Associative array operations

4. Advanced Data Types (4/5)

- Associative array usage
- Associative array best practices
- Queues
- Queue syntax
- Queue operations
- Queue usage

4. Advanced Data Types (5/5)

- Queue best practices
- `advanced_data_types.sv`: Demonstrates structures, unions, arrays, queues (iverilog)
- `advanced_data_types_cpp.cpp`: C++ equivalent using STL containers

5. SystemVerilog Operators (1/4)

- Streaming Operators
- Streaming operator syntax
- Streaming operator usage
- Packing/unpacking
- Streaming patterns
- Set Membership Operators

5. SystemVerilog Operators (2/4)

- Inside operator
- Set membership
- Constraint usage
- Set operations
- SystemVerilog-Specific Operators
- Unique operator

5. SystemVerilog Operators (3/4)

- Priority operator
- SystemVerilog operators
- Operator usage
- Operator Overloading Concepts
- Overloading concepts
- Overloading patterns

5. SystemVerilog Operators (4/4)

- Overloading best practices

6. Packages and Namespaces (1/7)

- SystemVerilog packages → UVM packages (`uvm_pkg`)
- Shared definitions → UVM base classes and utilities
- Namespace management → UVM's `uvm_*` namespace
- Package organization → UVM library structure
- Package Organization
- Package syntax: `package name; ... endpackage`

6. Packages and Namespaces (2/7)

- Package declaration: Define shared items
- Package organization: Group related definitions
- Package best practices: Clear naming, logical grouping
- Shared Definitions
- Shared types: `typedef` declarations
- Shared functions: Reusable computation logic

6. Packages and Namespaces (3/7)

- Shared tasks: Reusable procedural code
- Shared constants: `parameter` and `const` declarations
- Package Import
- Wildcard import: `import pkg::*;` (imports all)
- Explicit import: `import pkg::name;` (imports specific)
- Fully qualified: `pkg::name` (no import needed)

6. Packages and Namespaces (4/7)

- Import scope: Module-level or package-level
- Namespace Management
- Avoid naming conflicts: Packages provide namespaces
- Explicit vs. wildcard: Trade-offs in clarity vs. convenience
- Namespace organization: Hierarchical package structure
- Namespace best practices: Clear package naming

6. Packages and Namespaces (5/7)

- Testbench Library Organization
- Library structure: Organize packages by functionality
- Library organization: Common utilities, types, functions
- Library reusability: Share across projects
- Library best practices: Documentation, versioning
- `testbench_pkg.sv`: Comprehensive package with:

6. Packages and Namespaces (6/7)

- Type definitions (struct)
- Function definitions (automatic functions)
- Task definitions (automatic tasks)
- Constants and parameters
- `package_example.sv`: Demonstrates package usage:
- Package import (wildcard and explicit)

6. Packages and Namespaces (7/7)

- Using package types, functions, tasks
- Namespace management

7. SystemVerilog Procedural Blocks (1/5)

- Always_comb, Always_ff, Always_latch
- Always_comb syntax
- Always_ff syntax
- Always_latch syntax
- Block usage
- Unique and Priority Case

7. SystemVerilog Procedural Blocks (2/5)

- Unique case syntax
- Priority case syntax
- Case usage
- Case best practices
- SystemVerilog-Specific Constructs
- SystemVerilog constructs

7. SystemVerilog Procedural Blocks (3/5)

- Construct usage
- Construct best practices
- Location: `module6/examples/sv_classes/class_based_t
  estbench.sv`
- Demonstrates: SystemVerilog classes, object-oriented
  testbenches
- Location: `module6/examples/randomization/randomized
  _testbench.sv`
- Demonstrates: Randomization, constraints, random
  test generation

7. SystemVerilog Procedural Blocks (4/5)

- Location: `module6/examples/interfaces/interface_bas
  ed_testbench.sv`
- Demonstrates: Interfaces, modports, interface-based
  testbenches
- Location: (Coming soon)
- Demonstrates: Transaction classes, transaction
  sequences
- Location: All examples
- Demonstrates: SystemVerilog features in testbenches

7. SystemVerilog Procedural Blocks (5/5)

- Location: `module6/examples/equivalent_cpp/`
- Demonstrates: C++ equivalents for SystemVerilog features

Hands-on examples

Module 6 self-check

```
$ ./scripts/module6.sh --help
```

Terminal: module6_check

```
Usage: ./scripts/module6.sh [OPTIONS]

Module 6: SystemVerilog Testbench Features
This script runs examples and tests for Module 6.
Note: Many SystemVerilog features work best with iverilog. Verilator has limitations.

OPTIONS:
  Examples:
    --sv-classes            Run SystemVerilog classes examples (iverilog)
    --randomization         Run randomization examples (iverilog)
    --interfaces            Run interface examples (iverilog)
    --advanced-data-types   Run advanced data types examples
    --packages              Run package examples (iverilog)
    --equivalent-cpp        Show C++ equivalent patterns
    --all-examples          Run all examples (default)
    --skip-examples         Skip all examples

  Tests:
    --iverilog-tests        Run iverilog testbenches
    --cpp-tests             Run C++ testbenches
    --all-tests             Run all tests (default)
    --skip-tests            Skip all tests

  Environment:
    --jobs N                Number of parallel build jobs (default: 8)
    --no-clean              Skip cleaning before build (faster rebuilds)

  Other:
    --help, -h              Show this help message

EXAMPLES:
  # Run all examples and tests
  ./scripts/module6.sh

  # Run only SystemVerilog classes examples
  ./scripts/module6.sh --sv-classes
```

Exercise scaffold

```
./scripts/module6.sh --scaffold
```

Demo: SV classes

```
$ cd module6/examples/sv_classes && make all
```

Terminal: ex01_sv_classes

```
=== Compiling SystemVerilog class-based testbench (iverilog) ===
iverilog -g2012 -o class_based_testbench class_based_testbench.sv ../../dut/simple_gates/and_gat
=== Running simulation ===
vvp class_based_testbench
========================================
Class-Based Testbench Example
========================================

--- Deterministic Tests ---
[PASS] Test 1: a=0, b=0, result=0, expected=0
[PASS] Test 1: a=0, b=1, result=0, expected=0
[PASS] Test 1: a=1, b=0, result=0, expected=0
[PASS] Test 1: a=1, b=1, result=1, expected=1

--- Randomized Tests (using transaction class) ---
Transaction: a=0, b=1, expected=0
[PASS] Test 1: a=0, b=1, result=0, expected=0
Transaction: a=1, b=1, expected=1
[PASS] Test 1: a=1, b=1, result=1, expected=1
Transaction: a=1, b=1, expected=1
[PASS] Test 1: a=1, b=1, result=1, expected=1
Transaction: a=1, b=0, expected=0
[PASS] Test 1: a=1, b=0, result=0, expected=0

========================================
Test Summary
========================================
Total tests:  1
Passed:       1
Failed:       0
========================================
========================================
class_based_testbench.sv:330: $finish called at 10000 (1ps)
=== Compiling C++ class-based testbench (Verilator equivalent) ===
verilator --cc --exe --build -I../../dut/simple_gates \
../../dut/simple_gates/and_gate.v class_based_testbench_cpp.cpp
```

Demo: Interfaces

```
$ cd module6/examples/interfaces && make all
```

Terminal: ex02_interfaces

```
=== Compiling SystemVerilog interface-based testbench (iverilog) ===
iverilog -g2012 -o interface_based_testbench interface_based_testbench.sv ../../dut/simple_gates
=== Running simulation ===
vvp interface_based_testbench
========================================
Interface-Based Testbench Example
========================================
VCD info: dumpfile interface_based_testbench.vcd opened for output.
[PASS] Test 1: a=0, b=0, y=0, expected=0
[PASS] Test 2: a=0, b=1, y=0, expected=0
[PASS] Test 3: a=1, b=0, y=0, expected=0
[PASS] Test 4: a=1, b=1, y=1, expected=1

========================================
Test Summary
========================================
Total tests:  4
Passed:       4
Failed:       0
========================================
✓ All tests PASSED!
========================================
interface_based_testbench.sv:307: $finish called at 30000 (1ps)
=== Compiling C++ interface-based testbench (Verilator equivalent) ===
verilator --cc --exe --build -I../../dut/simple_gates \
□../../dut/simple_gates/and_gate.v interface_based_testbench_cpp.cpp
make: verilator: No such file or directory
make: *** [Makefile:20: interface_based_testbench_cpp] Error 127
```

Demo: Randomization

```
$ cd module6/examples/randomization && make all
```

Terminal: ex03_randomization

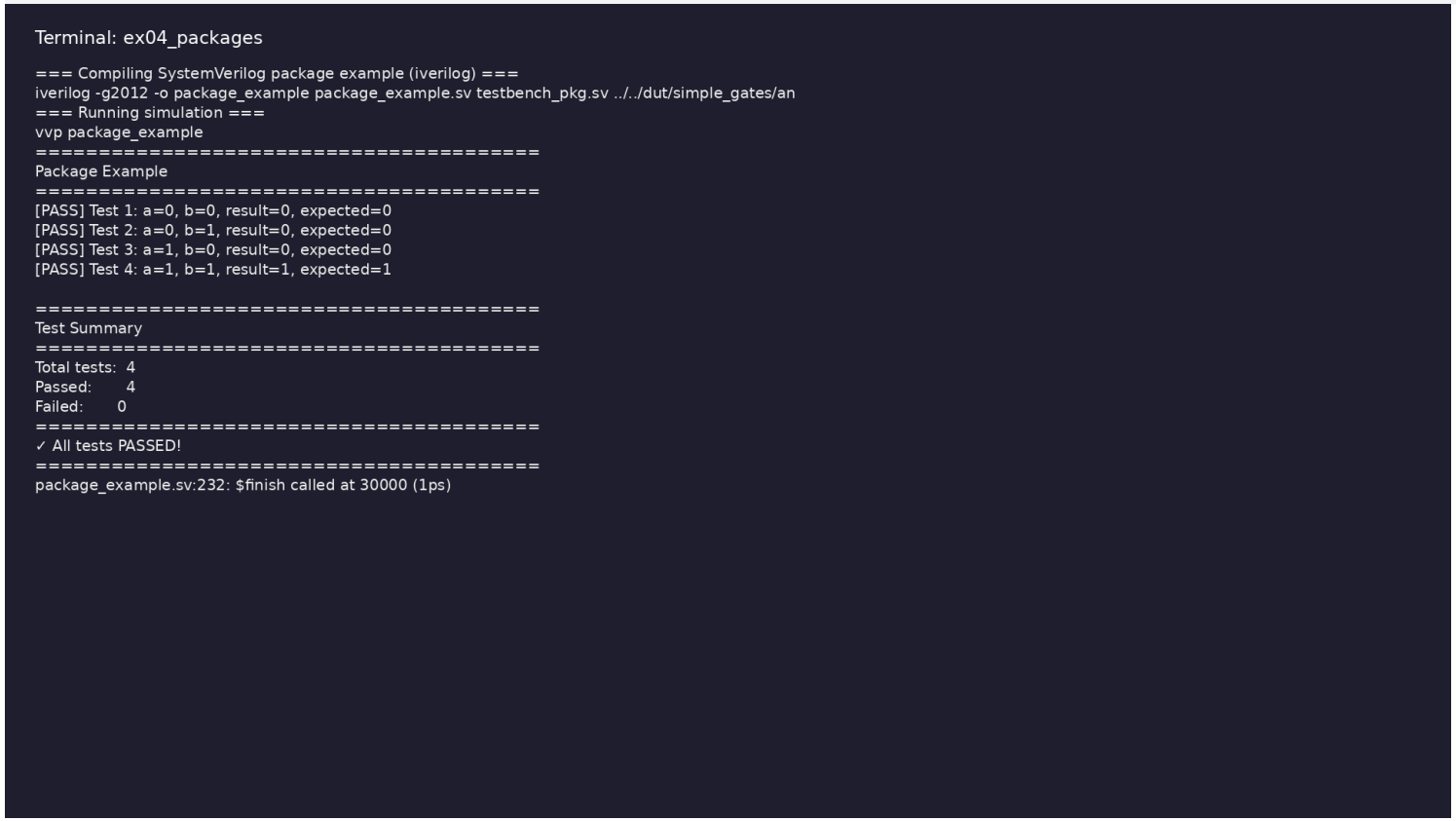
```
=== Compiling SystemVerilog randomized testbench (iverilog) ===
iverilog -g2012 -o randomized_testbench randomized_testbench.sv ../../dut/multiplexers/mux_4to1.
=== Running simulation ===
vvp randomized_testbench
========================================
Randomized Testbench Example
========================================
Random seed: 2450863396

Generating random test vectors:
Transaction: sel=01, in0=1, in1=1, in2=1, in3=1, expected=1
VCD info: dumpfile randomized_testbench.vcd opened for output.
  [PASS] Test 1: out=1, expected=1
Transaction: sel=01, in0=0, in1=1, in2=1, in3=0, expected=1
  [PASS] Test 2: out=1, expected=1
Transaction: sel=01, in0=1, in1=0, in2=1, in3=0, expected=0
  [PASS] Test 3: out=0, expected=0
Transaction: sel=01, in0=0, in1=1, in2=1, in3=0, expected=1
  [PASS] Test 4: out=1, expected=1
Transaction: sel=11, in0=0, in1=0, in2=0, in3=1, expected=1
  [PASS] Test 5: out=1, expected=1
Transaction: sel=00, in0=1, in1=1, in2=1, in3=1, expected=1
  [PASS] Test 6: out=1, expected=1
Transaction: sel=10, in0=0, in1=0, in2=0, in3=1, expected=0
  [PASS] Test 7: out=0, expected=0
Transaction: sel=10, in0=1, in1=1, in2=1, in3=1, expected=1
  [PASS] Test 8: out=1, expected=1
Transaction: sel=01, in0=0, in1=0, in2=1, in3=1, expected=0
  [PASS] Test 9: out=0, expected=0
Transaction: sel=11, in0=0, in1=0, in2=0, in3=0, expected=0
  [PASS] Test 10: out=0, expected=0

========================================
Test Summary
========================================
Total tests:  10
```

Demo: Packages

```
$ cd module6/examples/packages && make all
```



Demo: Equivalent C++

```
$ head -30 module6/examples/equivalent_cpp/verilator_limitations.md
```

Terminal: ex05_equivalent_cpp

```
# Verilator Limitations and C++ Alternatives

This document outlines SystemVerilog features that work well with iverilog but have limitations

## SystemVerilog Classes

### iverilog Support
- □ Full class support
- □ Inheritance
- □ Polymorphism
- □ Virtual methods

### Verilator Limitations
- □ Limited class support
- □ No inheritance
- □ No polymorphism

### C++ Alternative
Use C++ classes directly:
```cpp
class Transaction {
    // C++ class implementation
};
```

Randomization

iverilog Support
- □ `rand` and `randc` keywords
- □ Constraint blocks
```

Practice & assessment

What you should know (1/9)

- Use SystemVerilog classes in testbenches (iverilog)
- Implement randomization (iverilog)
- Use interfaces effectively (iverilog)
- Apply advanced data types
- Organize code with packages
- Write modern SystemVerilog testbenches

What you should know (2/9)

- Understand Verilator limitations and alternatives
- Use SystemVerilog classes
- Organize testbench with classes
- Compare with procedural approach
- Use rand variables
- Add constraints

What you should know (3/9)

- Generate random tests
- Define interfaces
- Use modports
- Connect DUT via interface
- Use classes for transactions
- Implement transaction sequences

What you should know (4/9)

- Verify transactions
- Create packages
- Share definitions
- Manage namespaces
- Use C++ classes
- Use C++ random number generators

What you should know (5/9)

- Use C++ structs for interfaces
- □ Full SystemVerilog class support
- □ Randomization support
- □ Interface support
- □ Package support
- □ Advanced data types

What you should know (6/9)

- × Limited SystemVerilog class support
- × Limited randomization support
- × Limited interface support
- × Limited package support
- □ Use C++ equivalents instead
- Can use SystemVerilog classes in testbenches (iverilog)

What you should know (7/9)

- Can implement randomization (iverilog)
- Can use interfaces effectively (iverilog)
- Can apply advanced data types
- Can organize code with packages
- Can write modern SystemVerilog testbenches
- Can understand Verilator limitations and alternatives

What you should know (8/9)

- Module 7: Coverage and Assertions - Master coverage and assertion-based verification
- Module 8: Verification Methodology and Best Practices - Learn industry best practices
- UVM Core Repository: https://github.com/universal-verification-methodology/core
- Official UVM implementation
- Examples and testbenches
- Documentation and best practices

What you should know (9/9)

- Reference implementations of UVM patterns

Assessment checklist

- Can use SystemVerilog classes in testbenches (iverilog)
- Can implement randomization (iverilog)
- Can use interfaces effectively (iverilog)
- Can apply advanced data types
- Can organize code with packages
- Can write modern SystemVerilog testbenches

Summary & next steps

- Master SystemVerilog features for advanced testbench development (primarily for iverilog, with Veri…
- Complete module6/CHECKLIST.md
- Review module6/EXAMPLES.md and run each lab
- Next: Next module in course